

4강

데이터 수집

데이터 분석의 첫 걸음 — 데이터를 어떻게 가져올까?

CSV / Excel 파일

웹 스크래핑

requests + BeautifulSoup

강의 시간: 약 40분 | Python을 이용한 데이터 분석

01

데이터 수집의 이해

데이터 분석 파이프라인에서 수집 단계의
역할과 다양한 수집 방법을 이해한다

02

파일 데이터 불러오기

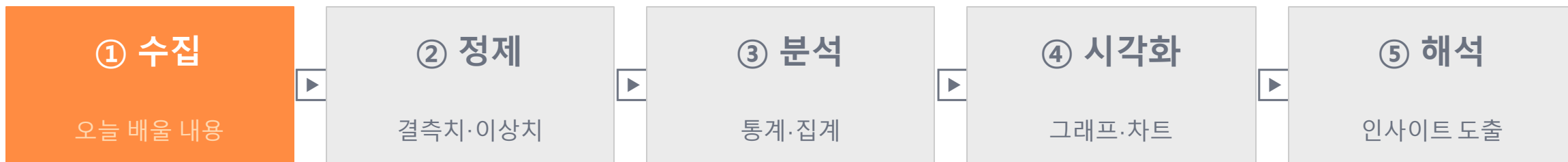
pandas로 CSV와 Excel 파일을
불러오고 기본 탐색을 할 수 있다.

03

웹 스크래핑

requests와 BeautifulSoup으로
웹 페이지의 데이터를 추출할 수 있다.

데이터 분석 파이프라인



왜 데이터 수집이 중요한가?



데이터 수집 방법 종류

오늘 다룰 내용

파일 (CSV · Excel · JSON)

오늘 학습!

가장 일반적인 방법
이미 정리된 데이터를 불러오기
→ `pd.read_csv()` / `pd.read_excel()`

웹 스크래핑

오늘 학습!

웹 페이지에서 직접 데이터 추출
가격, 뉴스, 날씨, 주가 등 수집
→ `requests` + `BeautifulSoup`

API (Application Programming Interface)

서비스가 제공하는 공식 데이터 채널
공공데이터포털, 카카오, 네이버 등
→ `requests` + JSON 파싱

데이터베이스 (DB)

MySQL, PostgreSQL, SQLite 등
기업 내부 데이터 접근
→ `pandas` + `SQLAlchemy`

CSV 파일이란?

CSV (Comma-Separated Values)

쉼표(,)로 값을 구분하는 텍스트 형식의 데이터 파일

- 첫 번째 줄(헤더): 컬럼 이름
- 이후 줄: 실제 데이터 (한 줄 = 한 행)
- 메모장으로도 열 수 있는 단순한 형식
- 엑셀, Python, R 등 모든 도구 호환
- Excel과 다르게 수식·서식 없음 — 순수 데이터만

⚠ 인코딩 주의!

한글이 포함된 파일은 인코딩 설정 필수
UTF-8: 국제 표준 / EUC-KR (CP949): 오래된 한국 파일
→ 잘못된 인코딩으로 열면 한글이 깨짐!

▶ CSV 파일 내부 구조

```
이름,국어,영어,수학,학점  
홍길동,85,92,78,B  
김철수,90,88,95,A  
이영희,72,68,75,C  
박민준,88,91,83,B  
최서연,95,97,99,A
```

↑ 헤더 (컬럼 이름)

↓ 데이터 행 (5개 학생)

pandas DataFrame으로 변환하면:

	이름	국어	영어	수학	학점
0	홍길동	85	92	78	B
1	김철수	90	88	95	A
2	이영희	72	68	75	C

pd.read_csv() — CSV 파일 불러오기

📁 파일 수집

🕒 10분

기본 사용법

```
import pandas as pd

# ① 기본 불러오기
df = pd.read_csv("students.csv")

# ② 한글 파일 — UTF-8
df = pd.read_csv("students.csv",
                  encoding="utf-8")

# ③ 오래된 한글 파일 — EUC-KR
df = pd.read_csv("students.csv",
                  encoding="euc-kr")

print(df.head())
```

핵심 포인트

```
import pandas as pd
```

pandas를 pd 라는 별명으로 불러오기
관습적으로 pd 사용 — 항상 첫 줄에!

```
pd.read_csv("파일명")
```

파일명.csv 또는 경로 포함 가능
예) "C:/data/students.csv"

encoding 파라미터

한글 깨짐 시 반드시 지정
utf-8 먼저 시도 → 안 되면 euc-kr

pd.read_csv() — 주요 파라미터

📁 파일 수집

자주 쓰는 파라미터를 알면 어떤 파일이든 불러올 수 있습니다.

파라미터	설명	예시
encoding	파일 인코딩 지정	"utf-8" / "euc-kr"
sep	구분자 (기본값: 쉼표)	sep="\t" (탭 구분)
header	헤더 행 번호 (기본값: 0)	header=None (헤더 없음)
index_col	인덱스로 쓸 컬럼 번호/이름	index_col=0 또는 "이름"
usecols	불러올 컬럼 선택	usecols=["이름", "점수"]
nrows	불러올 최대 행 수	nrows=100
skiprows	건너뛴 행 수	skiprows=2
na_values	결측치로 처리할 값	na_values=["N/A", "-"]

pd.read_excel() & 불러온 데이터 확인

📁 파일 수집

Excel 파일 불러오기

```
import pandas as pd

# 기본 불러오기 (첫 번째 시트)
df = pd.read_excel("students.xlsx")

# 시트 이름으로 지정
df = pd.read_excel("students.xlsx",
                  sheet_name="성적표")

# 여러 시트 한꺼번에 불러오기
all_sheets = pd.read_excel("data.xlsx",
                          sheet_name=None)
df1 = all_sheets["Sheet1"]
```

💡 CSV vs Excel

openpyxl 설치 필요: `pip install openpyxl`

불러온 데이터 확인 필수 3종 세트

`df.head()`

앞 5행 미리보기 (기본값)

`df.tail(3)`

뒤 N행 미리보기

`df.shape`

(행 수, 열 수) 튜플 반환

`df.columns`

컬럼 이름 목록

`df.info()`

타입·결측치 한눈에 확인 ★

`df.describe()`

수치형 컬럼 기초 통계

웹 스크래핑 (Web Scraping) 이란?

 웹 스크래핑

 20분

웹 스크래핑이란?

웹 브라우저가 웹 페이지를 렌더링하듯, 코드로 웹 페이지의 HTML을 받아와서 원하는 데이터만 추출하는 기술

어디에 활용하나요?



쇼핑몰 가격 비교

여러 쇼핑몰의 제품 가격을
자동으로 수집해 비교



뉴스 수집

특정 키워드 뉴스를
자동으로 모아 분석



주가·환율

실시간 금융 데이터를
주기적으로 수집



날씨 데이터

날씨 정보를 수집해
외부 요인 분석에 활용

오늘 사용할 도구: requests (웹 페이지 요청) + BeautifulSoup (HTML 파싱 및 데이터 추출)

HTML 구조 이해 — 스크래핑 전 필수 지식

🔧 웹 스크래핑

웹 페이지는 HTML(HyperText Markup Language)로 이루어져 있습니다.

```
<html>
  <body>
    <h1>학생 성적표</h1>
    <table class="score-table">
      <tr>
        <th>이름</th> <th>점수</th> <th>학점</th>
      </tr>
      <tr>
        <td>홍길동</td> <td>85</td> <td>B</td>
      </tr>
      <tr>
        <td>김철수</td> <td>92</td> <td>A</td>
      </tr>
    </table>
  </body>
</html>
```

핵심 HTML 태그

<code><table></code>	표 전체를 감싸는 태그
<code><tr></code>	테이블 행 (Table Row)
<code><th></code>	헤더 셀 (굵게 표시)
<code><td></code>	데이터 셀 (Table Data)
<code><div></code>	영역을 나누는 범용 태그
<code></code>	인라인 텍스트 그룹
<code>class="..."</code>	CSS 식별자 (스크래핑에 중요!)
<code>id="..."</code>	유일한 요소 식별자

💡 개발자 도구 (F12) → Elements 탭에서 원하는 요소의 태그/클래스 확인!

requests — 웹 페이지 요청하기

🛠 웹 스크래핑

requests란?

Python에서 웹 브라우저처럼 URL에 HTTP 요청을 보내
서버로부터 HTML 응답을 받아오는 라이브러리

HTTP 상태 코드

200 OK — 요청 성공 ✓

403 Forbidden — 접근 거부 (로봇 차단)

404 Not Found — 페이지 없음

500 Internal Server Error — 서버 오류

```
# 설치 (터미널에서 한 번만)
# pip install requests

import requests

url = "https://example.com"
response = requests.get(url)

# 상태 코드 확인 (200 = 성공)
print(response.status_code)      # 200

# HTML 내용 가져오기
html = response.text
print(html[:300])                # 앞 300글자 미리보기

# 한글 인코딩 명시적 설정
response.encoding = "utf-8"
html = response.text

# 헤더 추가 (봇 차단 우회)
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get(url, headers=headers)
```

BeautifulSoup — HTML 파싱 & 데이터 추출

🌐 웹 스크래핑

BeautifulSoup이란?

HTML 문자열을 파싱(분석)하여 원하는 태그의 데이터를 쉽게 꺼낼 수 있도록 해주는 라이브러리

핵심 메서드

<code>find(태그)</code>	첫 번째 일치 태그 반환
<code>find_all(태그)</code>	일치하는 모든 태그 리스트
<code>find("div", class_="abc")</code>	클래스 속성으로 탐색
<code>태그.text</code>	태그 안의 텍스트 추출
<code>태그.get("href")</code>	속성값 추출 (링크 주소 등)
<code>태그.find_all("td")</code>	하위 태그 탐색

```
# 설치: pip install beautifulsoup4
from bs4 import BeautifulSoup
import requests

# 1단계: HTML 가져오기
url = "https://example.com"
html = requests.get(url).text

# 2단계: HTML 파싱
soup = BeautifulSoup(html, "html.parser")

# 3단계: 원하는 데이터 추출
# 태그 하나 찾기
title = soup.find("h1")
print(title.text)      # 텍스트만 추출

# 여러 태그 찾기
items = soup.find_all("li")
for item in items:
    print(item.text)

# class 속성으로 찾기
price = soup.find("span", class_="price")
print(price.text)
```

실전 예시 — 테이블 데이터 스크래핑

🌐 웹 스크래핑

☆ 핵심 패턴

전체 흐름 코드

```
import requests, pandas as pd
from bs4 import BeautifulSoup

# ① 웹 페이지 요청
url = "https://example.com/scores"
resp = requests.get(url)
resp.encoding = "utf-8"

# ② HTML 파싱
soup = BeautifulSoup(resp.text, "html.parser")

# ③ 테이블 찾기
table = soup.find("table")
rows = table.find_all("tr")

# ④ 데이터 추출
data = []
for row in rows[1:]: # 헤더(0번) 제외
    cells = row.find_all("td")
    if cells:
        data.append(
            [c.text.strip() for c in cells])

# ⑤ DataFrame 변환
df = pd.DataFrame(data,
    columns=["이름", "점수", "학점"])
print(df)
```

단계별 설명

①

요청 (requests.get)

URL에 GET 요청 → HTML 문자열 수신

②

파싱 (BeautifulSoup)

HTML 문자열을 트리 구조로 변환

③

태그 탐색 (find)

원하는 table, tr, td 태그 찾기

④

텍스트 추출

.text.strip() 으로 공백 제거하며 추출

⑤

DataFrame 변환

리스트 → pd.DataFrame() → 분석 준비 완료

💡 팁: pd.read_html(url) 한 줄로 테이블 자동 수집 가능!

수집한 데이터 저장하기

📁 저장

수집한 데이터를 파일로 저장해 두면 다음번에 다시 수집하지 않아도 됩니다.

CSV로 저장 — df.to_csv()

```
# 기본 저장
df.to_csv("output.csv", index=False)

# 한글 포함 시 — 엑셀에서 깨짐 방지
df.to_csv("output.csv",
          index=False,
          encoding="utf-8-sig")

# 특정 컬럼만 저장
df[["이름", "점수"]].to_csv("score.csv",
                           index=False)
```

⚠ index=False 를 꼭 써야 하는 이유

기본값(index=True)이면 0,1,2... 숫자 인덱스 열이 추가됨
→ 나중에 다시 불러올 때 불필요한 컬럼이 생김

Excel로 저장 — df.to_excel()

```
# 기본 저장
df.to_excel("output.xlsx", index=False)

# 시트 이름 지정
df.to_excel("output.xlsx",
            sheet_name="성적표",
            index=False)

# 여러 시트로 저장
with pd.ExcelWriter("multi.xlsx") as wr:
    df1.to_excel(wr, sheet_name="1반")
    df2.to_excel(wr, sheet_name="2반")
```

CSV vs Excel 언제 쓸까?

CSV: 용량 작고 범용적, 여러 도구와 호환 → 데이터 교환에 적합
Excel: 서식·다중시트 필요할 때, 비개발자와 공유 시

웹 스크래핑 — 윤리 & 주의사항

⚠ 반드시 숙지

스크래핑은 강력한 도구입니다. 올바르게 사용하세요.

robots.txt 확인

모든 사이트는 robots.txt 파일로 크롤링 허용 범위를 명시합니다.
브라우저에서 "사이트주소/robots.txt" 에 접속해 확인하세요.
Disallow: / → 스크래핑 금지!

요청 간격 두기 (time.sleep)

짧은 시간에 너무 많은 요청을 보내면 서버에 부하를 줍니다.
import time 후 time.sleep(1) 로 1초 간격 필수!
안 지키면 IP가 차단될 수 있습니다.

저작권 & 이용약관

수집한 데이터를 상업적으로 이용하면 저작권 침해가 될 수 있습니다.
각 사이트 이용약관의 "데이터 수집/크롤링" 조항을 확인하세요.
개인 학습·연구 목적은 대체로 허용됩니다.

개인정보 주의

개인을 식별할 수 있는 정보(이름, 이메일 등)를 무단 수집·공개하면
개인정보보호법 위반이 될 수 있습니다.
공개된 데이터라도 목적과 범위에 주의하세요.

꿀팁 — pd.read_html() & 전체 흐름 정리

pd.read_html() — 테이블 스크래핑 한 줄 완성!

HTML 페이지에서 <table> 태그를 자동으로 찾아
DataFrame 리스트로 반환하는 pandas 내장 기능

```
import pandas as pd

# URL에서 모든 테이블 자동 수집
tables = pd.read_html("https://example.com")

# 첫 번째 테이블 선택
df = tables[0]
print(df.head())

# requests로 가져온 HTML에서도 사용 가능
import requests
html = requests.get(url).text
tables = pd.read_html(html)
df = tables[0]
```

✅ 장점: 코드 단 2줄 / ⚠️ 한계: <table> 태그 없는 페이지는 BeautifulSoup 필요

오늘 배운 전체 흐름

1

URL 결정

수집할 웹 페이지 또는 파일 경로 확인

2

데이터 수신

requests.get(url) 또는 pd.read_csv()

3

파싱 & 추출

BeautifulSoup.find_all() 또는 직접 로드

4

DataFrame 변환

pd.DataFrame(data) → 컬럼 지정

5

확인 & 저장

df.head() / df.info() / df.to_csv()

이론 정리 & 다음 시간 예고

오늘 배운 핵심 내용

CSV / Excel 불러오기

- `pd.read_csv("파일명", encoding="utf-8")`
- `pd.read_excel("파일명", sheet_name=0)`
- `df.head()` / `df.info()` / `df.describe()` 로 확인

웹 스크래핑

- `requests.get(url).text` → HTML 가져오기
- `BeautifulSoup(html, "html.parser")` → 파싱
- `find()` / `find_all("td")` → 데이터 추출

데이터 저장

- `df.to_csv("output.csv", index=False)`
- `df.to_excel("output.xlsx", index=False)`
- `index=False` 필수! / 한글 → utf-8-sig

다음 시간: 실습 + 5강 예고

실습: 실제 CSV 파일 불러오기 / 공공 데이터 웹 스크래핑 실습
5강 예고: 수집한 데이터의 결측치·이상치 처리 + 데이터 시각화